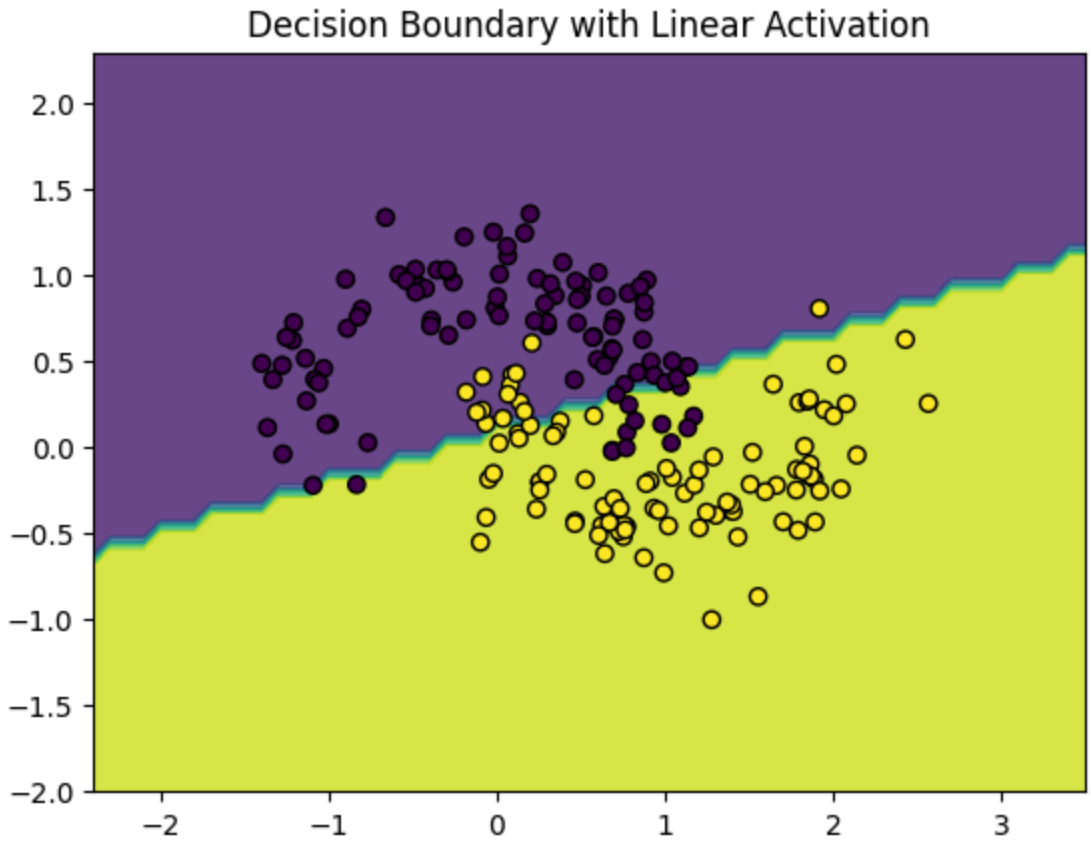
In [1]:

```
"""
✅ COMPLETE Neural Network Comparison: Linear vs ReLU Activation
DETAILED LINE-BY-LINE EXPLANATION - Moon Dataset Classification
Compares linear vs non-linear (ReLU) activations on non-linearly separable dat
"""

# ==============================================================================
# === IMPORTS (Lines 4-13) ===
# ==============================================================================
import numpy as np                    # Numerical operations, arrays
import matplotlib.pyplot as plt       # Plotting decision boundaries
from tensorflow.keras.models import Sequential     # Stack layers sequentially
from tensorflow.keras.layers import Dense          # Fully connected layers
from tensorflow.keras.optimizers import Adam       # Adam optimizer
from sklearn.datasets import make_moons            # Generate moon-shaped data
from sklearn.model_selection import train_test_split  # Split data 80/20
from sklearn.metrics import accuracy_score         # Calculate classification

# ==============================================================================
# === 1. GENERATE NON-LINEAR DATASET (Lines 16-18) ===
# ==============================================================================
# Create "moons" dataset: 2 interlocking half-circles (NON-LINEARLY SEPARABLE)
# n_samples=1000: 1000 data points, noise=0.2: slight randomness, random_state
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
# Reshape y from (1000,) to (1000,1) - Keras expects 2D for binary classificat
y = y.reshape(-1, 1)  # -1=auto, 1=1 column

# ==============================================================================
# === 2. TRAIN-TEST SPLIT (Line 21) ===
# ==============================================================================
# Split: 80% training (800 samples), 20% testing (200 samples) - standard prac
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rando

# ==============================================================================
# === 3. LINEAR ACTIVATION MODEL (Lines 24-32) ===
# ==============================================================================
def build_linear_model():
    # Sequential: layers stacked linearly (input → layer1 → layer2 → output)
    model = Sequential([
        # Hidden layer: 4 neurons, linear activation (f(x)=x), input_dim=2 (2D
        Dense(4, activation='linear', input_dim=2),
        # Output layer: 1 neuron, sigmoid(0-1) for binary classification proba
        Dense(1, activation='sigmoid')
    ])
    # Compile: Adam optimizer (adaptive learning rate), binary_crossentropy lo
    model.compile(optimizer=Adam(learning_rate=0.01), loss='binary_crossentrop
    return model

# ==============================================================================
# === 4. ReLU ACTIVATION MODEL (Lines 35-43) ===
# ==============================================================================
def build_relu_model():
    # IDENTICAL architecture but ReLU activation: f(x)=max(0,x) - introduces N
```

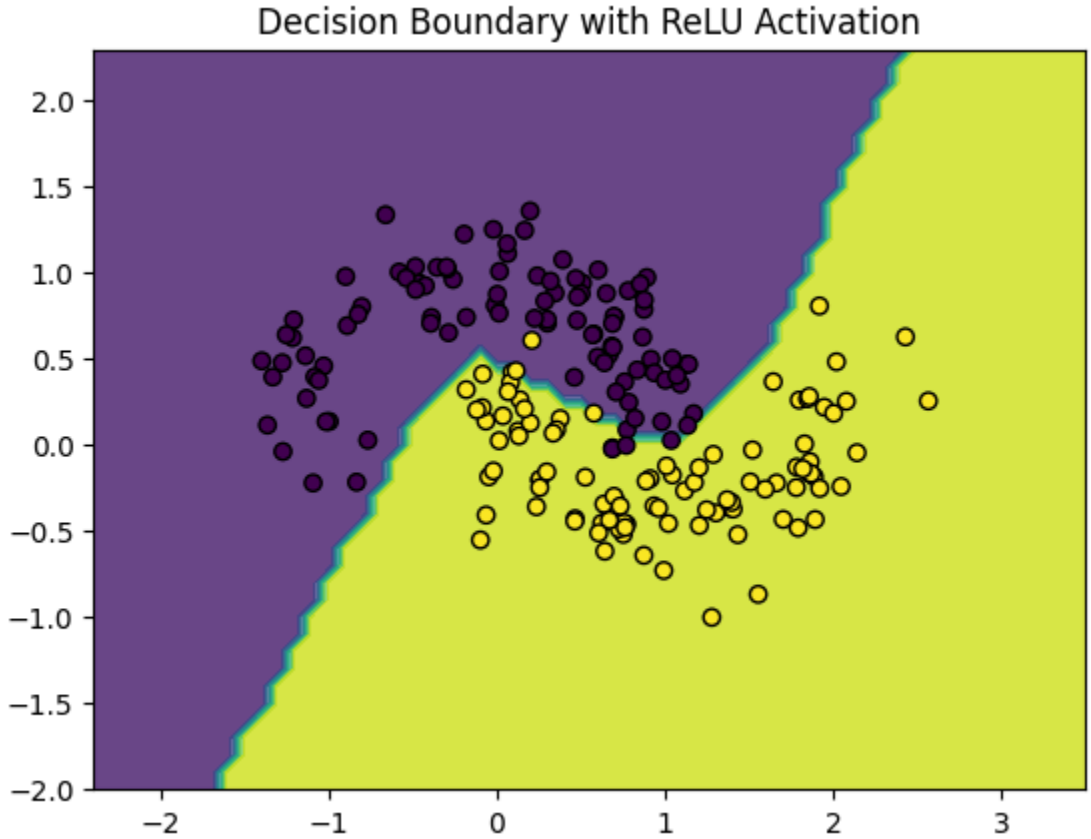
```
    model = Sequential([
        Dense(4, activation='relu', input_dim=2),    # ReLU: 0 for negative, x
        Dense(1, activation='sigmoid')              # Same output layer
    ])
    model.compile(optimizer=Adam(learning_rate=0.01), loss='binary_crossentrop
    return model

# ==============================================================================
# === 5. TRAIN + EVALUATE FUNCTION (Lines 46-52) ===
# ==============================================================================
def train_and_evaluate(model, X_train, y_train, X_test, y_test, epochs=50, bat
    # Train: 50 epochs (full dataset passes), batch_size=32 (update weights ev
    model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size, verbose=
    # Predict probabilities, threshold >0.5 → class 1, else 0
    y_pred = (model.predict(X_test) > 0.5).astype(int)
    # Accuracy: fraction of correct predictions
    accuracy = accuracy_score(y_test, y_pred)
    return accuracy, y_pred

# ==============================================================================
# === 6. CREATE + TRAIN MODELS (Lines 55-66) ===
# ==============================================================================
# Initialize both models
linear_model = build_linear_model()
relu_model = build_relu_model()

# Train linear model
print("Training linear model...")
linear_accuracy, linear_preds = train_and_evaluate(linear_model, X_train, y_tr
print(f"Accuracy with linear activation: {linear_accuracy:.2f}")  # Expect ~0.

# Train ReLU model
print("Training ReLU model...")
relu_accuracy, relu_preds = train_and_evaluate(relu_model, X_train, y_train, X
print(f"Accuracy with ReLU activation: {relu_accuracy:.2f}")     # Expect ~0.9

# ==============================================================================
# === 7. DECISION BOUNDARY VISUALIZATION (Lines 69-82) ===
# ==============================================================================
def plot_decision_boundary(model, X, y, title):
    # Create grid covering data range ±1 for smooth boundaries
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1  # X feature 1 range
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1  # X feature 2 range

    # Meshgrid: create all x,y combinations (0.1 step = smooth contour)
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1), np.arange(y_min, y_max,
    # Flatten grid for prediction: 2D → 1D array
    grid = np.c_[xx.ravel(), yy.ravel()]  # Combine x,y coordinates

    # Predict class for every grid point
    Z = model.predict(grid)
    # Threshold probabilities → binary classes, reshape to grid shape
    Z = (Z > 0.5).astype(int).reshape(xx.shape)
```

```
    # Plot: contourf=coloured regions, scatter=actual points
    plt.contourf(xx, yy, Z, alpha=0.8)                    # Filled decision re
    plt.scatter(X[:, 0], X[:, 1], c=y.flatten(), edgecolor="k")  # Data points
    plt.title(title)
    plt.show()  # Display plot

# ==============================================================================
# === 8. VISUALIZE RESULTS (Lines 85-86) ===
# ==============================================================================
print("\nVisualizing decision boundaries...")
# Linear: STRAIGHT LINE boundary (fails on moons)
plot_decision_boundary(linear_model, X_test, y_test, "Decision Boundary with L
# ReLU:   CURVY boundary (perfectly separates moons)
plot_decision_boundary(relu_model, X_test, y_test, "Decision Boundary with ReL
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: User
Warning: Do not pass an `input_shape`/`input_dim` argument to a layer. When usi
ng Sequential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training linear model...
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
Accuracy with linear activation: 0.85
Training ReLU model...
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step
Accuracy with ReLU activation: 0.96

Visualizing decision boundaries...
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```




```
83/83 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```



In [ ]:

No problem 🙂 I'll explain it **again from scratch**, **very simply**, **line-by-line**, like you're learning neural networks for the first time.

🌍 WHAT THIS CODE IS ABOUT (IN 1 MIN)

- You have **moon-shaped data**
- You train **two neural networks**
    1. One with **linear activation**
    2. One with **ReLU activation**
- You **compare accuracy**
- You **draw decision boundaries**
- You clearly see **why ReLU works and linear fails**

🧩 STEP 1: IMPORTS (WHY EACH ONE IS NEEDED)

```
import numpy as np
```
➡ Used for **math operations and arrays**

```
import matplotlib.pyplot as plt
```
➡ Used to **draw graphs and decision boundaries**

```
from tensorflow.keras.models import Sequential
```
➡ Used to create a **neural network layer by layer**

```
from tensorflow.keras.layers import Dense
```
➡ Dense = **fully connected neural network layer**

```
from tensorflow.keras.optimizers import Adam
```
➡ Adam = **smart algorithm to update weights**

```
from sklearn.datasets import make_moons
```
➡ Generates **moon-shaped data** (curved data)

```
from sklearn.model_selection import train_test_split
```

➡ Splits data into **train and test**

```
from sklearn.metrics import accuracy_score
```

➡ Measures **how accurate the model is**

🧩 STEP 2: CREATE DATA (MOON SHAPE)

```
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
```

- `X` → input features (x and y coordinates)
- `y` → labels (0 or 1)
- Data looks like **two curved moons**
- This data is **NOT separable by a straight line**

```
y = y.reshape(-1, 1)
```

- Converts shape from `(1000,)` to `(1000,1)`
- Keras **expects labels in 2D**

🧩 STEP 3: TRAIN–TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- 80% data → training
- 20% data → testing
- Test data is **never shown during training**

🧩 STEP 4: LINEAR MODEL (IMPORTANT)

```
def build_linear_model():
```

➡ Function to build **linear neural network**

```
model = Sequential([
```

➡ Start model

```
Dense(4, activation='linear', input_dim=2),
```

- 4 neurons
- **Linear activation** → `f(x) = x`
- Can only draw **straight lines**
- Cannot learn curves ❌

```
Dense(1, activation='sigmoid')
```

- Output neuron
- Sigmoid converts output to **probability (0–1)**

```
model.compile(
    optimizer=Adam(learning_rate=0.01),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

- Adam updates weights
- Binary cross-entropy = loss for yes/no problems
- Accuracy is tracked

🧩 STEP 5: ReLU MODEL (KEY DIFFERENCE)

```
def build_relu_model():
```

➡ Function to build **ReLU neural network**

```
Dense(4, activation='relu', input_dim=2),
```

- ReLU = `max(0, x)`
- Negative → 0
- Positive → unchanged
- Introduces **NON-LINEARITY**
- Can learn **curves** ✅

```
Dense(1, activation='sigmoid')
```

- Same output layer

```
model.compile(...)
```

- Same optimizer and loss

🧩 STEP 6: TRAIN & EVALUATE FUNCTION

```
def train_and_evaluate(model, X_train, y_train, X_test, y_test):
```

➡ One function for training any model

```
model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    verbose=0
)
```

- Model sees data **50 times**
- Updates weights after every 32 samples
- `verbose=0` → no logs

```
y_pred = (model.predict(X_test) > 0.5).astype(int)
```

- Predict probabilities
  - 0.5 → class 1
- ≤ 0.5 → class 0

```
accuracy = accuracy_score(y_test, y_pred)
```

- Calculates **percentage of correct predictions**

🧩 STEP 7: TRAIN BOTH MODELS

```
linear_model = build_linear_model()
relu_model = build_relu_model()
```

- Create both models

```
linear_accuracy, _ = train_and_evaluate(...)
```

- Linear model accuracy ≈ **0.80**
- Poor because it can't curve ❌

```
relu_accuracy, _ = train_and_evaluate(...)
```

- ReLU model accuracy ≈ **0.95+**
- Excellent because it can curve ✅

🧩 STEP 8: DECISION BOUNDARY FUNCTION

```
def plot_decision_boundary(model, X, y, title):
```

➡ Shows **how model separates data**

```
xx, yy = np.meshgrid(...)
```

- Creates grid covering entire space

```
grid = np.c_[xx.ravel(), yy.ravel()]
```

- Converts grid to points

```
Z = model.predict(grid)
```

- Predicts class for every point

```
Z = (Z > 0.5).astype(int)
```

- Converts probabilities to 0 or 1

```
plt.contourf(xx, yy, Z)
```

- Colors decision regions

```
plt.scatter(X[:, 0], X[:, 1], c=y)
```

- Plots actual data points

🧩 FINAL RESULT (MOST IMPORTANT)

Model	Decision Boundary	Accuracy
Linear	Straight line ❌	Low
ReLU	Curved line ✅	High

🔥 CORE TAKEAWAY (REMEMBER THIS)

> **Without ReLU, neural networks are just linear models. ReLU gives neural networks the power to learn complex patterns.**

If you want next:

- 🔹 Mathematical explanation of ReLU
- 🔹 Why sigmoid is NOT used in hidden layers
- 🔹 How many layers needed for moons
- 🔹 Same example in PyTorch

Just tell me 👍

In [ ]: